

23	All-Pairs Shortest Paths	646
23.1	Shortest paths and matrix multiplication	648
23.2	The Floyd-Warshall algorithm	655
23.3	Johnson's algorithm for sparse graphs	662
24	Maximum Flow	670
24.1	Flow networks	671
24.2	The Ford-Fulkerson method	676
24.3	Maximum bipartite matching	693
25	Matchings in Bipartite Graphs	704
25.1	Maximum bipartite matching (revisited)	705
25.2	The stable-marriage problem	716
25.3	The Hungarian algorithm for the assignment problem	723

VII Selected Topics

	Introduction	745
26	Parallel Algorithms	748
26.1	The basics of fork-join parallelism	750
26.2	Parallel matrix multiplication	770
26.3	Parallel merge sort	775
27	Online Algorithms	791
27.1	Waiting for an elevator	792
27.2	Maintaining a search list	795
27.3	Online caching	802
28	Matrix Operations	819
28.1	Solving systems of linear equations	819
28.2	Inverting matrices	833
28.3	Symmetric positive-definite matrices and least-squares approximation	838
29	Linear Programming	850
29.1	Linear programming formulations and algorithms	853
29.2	Formulating problems as linear programs	860
29.3	Duality	866
30	Polynomials and the FFT	877
30.1	Representing polynomials	879
30.2	The DFT and FFT	885
30.3	FFT circuits	894

31	Number-Theoretic Algorithms	903
31.1	Elementary number-theoretic notions	904
31.2	Greatest common divisor	911
31.3	Modular arithmetic	916
31.4	Solving modular linear equations	924
31.5	The Chinese remainder theorem	928
31.6	Powers of an element	932
31.7	The RSA public-key cryptosystem	936
★ 31.8	Primality testing	942
32	String Matching	957
32.1	The naive string-matching algorithm	960
32.2	The Rabin-Karp algorithm	962
32.3	String matching with finite automata	967
★ 32.4	The Knuth-Morris-Pratt algorithm	975
32.5	Suffix arrays	985
33	Machine-Learning Algorithms	1003
33.1	Clustering	1005
33.2	Multiplicative-weights algorithms	1015
33.3	Gradient descent	1022
34	NP-Completeness	1042
34.1	Polynomial time	1048
34.2	Polynomial-time verification	1056
34.3	NP-completeness and reducibility	1061
34.4	NP-completeness proofs	1072
34.5	NP-complete problems	1080
35	Approximation Algorithms	1104
35.1	The vertex-cover problem	1106
35.2	The traveling-salesperson problem	1109
35.3	The set-covering problem	1115
35.4	Randomization and linear programming	1119
35.5	The subset-sum problem	1124

VIII Appendix: Mathematical Background

	Introduction	1139
A	Summations	1140
A.1	Summation formulas and properties	1140
A.2	Bounding summations	1145

B	Sets, Etc.	1153
B.1	Sets	1153
B.2	Relations	1158
B.3	Functions	1161
B.4	Graphs	1164
B.5	Trees	1169
C	Counting and Probability	1178
C.1	Counting	1178
C.2	Probability	1184
C.3	Discrete random variables	1191
C.4	The geometric and binomial distributions	1196
★ C.5	The tails of the binomial distribution	1203
D	Matrices	1214
D.1	Matrices and matrix operations	1214
D.2	Basic matrix properties	1219

Bibliography 1227

Index 1251

Preface

Not so long ago, anyone who had heard the word “algorithm” was almost certainly a computer scientist or mathematician. With computers having become prevalent in our modern lives, however, the term is no longer esoteric. If you look around your home, you’ll find algorithms running in the most mundane places: your microwave oven, your washing machine, and, of course, your computer. You ask algorithms to make recommendations to you: what music you might like or what route to take when driving. Our society, for better or for worse, asks algorithms to suggest sentences for convicted criminals. You even rely on algorithms to keep you alive, or at least not to kill you: the control systems in your car or in medical equipment.¹ The word “algorithm” appears somewhere in the news seemingly every day.

Therefore, it behooves you to understand algorithms not just as a student or practitioner of computer science, but as a citizen of the world. Once you understand algorithms, you can educate others about what algorithms are, how they operate, and what their limitations are.

This book provides a comprehensive introduction to the modern study of computer algorithms. It presents many algorithms and covers them in considerable depth, yet makes their design accessible to all levels of readers. All the analyses are laid out, some simple, some more involved. We have tried to keep explanations clear without sacrificing depth of coverage or mathematical rigor.

Each chapter presents an algorithm, a design technique, an application area, or a related topic. Algorithms are described in English and in a pseudocode designed to be readable by anyone who has done a little programming. The book contains 231 figures—many with multiple parts—illustrating how the algorithms work. Since we emphasize *efficiency* as a design criterion, we include careful analyses of the running times of the algorithms.

¹ To understand many of the ways in which algorithms influence our daily lives, see the book by Fry [162].

The text is intended primarily for use in undergraduate or graduate courses in algorithms or data structures. Because it discusses engineering issues in algorithm design, as well as mathematical aspects, it is equally well suited for self-study by technical professionals.

In this, the fourth edition, we have once again updated the entire book. The changes cover a broad spectrum, including new chapters and sections, color illustrations, and what we hope you'll find to be a more engaging writing style.

To the teacher

We have designed this book to be both versatile and complete. You should find it useful for a variety of courses, from an undergraduate course in data structures up through a graduate course in algorithms. Because we have provided considerably more material than can fit in a typical one-term course, you can select the material that best supports the course you wish to teach.

You should find it easy to organize your course around just the chapters you need. We have made chapters relatively self-contained, so that you need not worry about an unexpected and unnecessary dependence of one chapter on another. Whereas in an undergraduate course, you might use only some sections from a chapter, in a graduate course, you might cover the entire chapter.

We have included 931 exercises and 162 problems. Each section ends with exercises, and each chapter ends with problems. The exercises are generally short questions that test basic mastery of the material. Some are simple self-check thought exercises, but many are substantial and suitable as assigned homework. The problems include more elaborate case studies which often introduce new material. They often consist of several parts that lead the student through the steps required to arrive at a solution.

As with the third edition of this book, we have made publicly available solutions to some, but by no means all, of the problems and exercises. You can find these solutions on our website, <http://mitpress.mit.edu/algorithms/>. You will want to check this site to see whether it contains the solution to an exercise or problem that you plan to assign. Since the set of solutions that we post might grow over time, we recommend that you check the site each time you teach the course.

We have starred (★) the sections and exercises that are more suitable for graduate students than for undergraduates. A starred section is not necessarily more difficult than an unstarred one, but it may require an understanding of more advanced mathematics. Likewise, starred exercises may require an advanced background or more than average creativity.

To the student

We hope that this textbook provides you with an enjoyable introduction to the field of algorithms. We have attempted to make every algorithm accessible and interesting. To help you when you encounter unfamiliar or difficult algorithms, we describe each one in a step-by-step manner. We also provide careful explanations of the mathematics needed to understand the analysis of the algorithms and supporting figures to help you visualize what is going on.

Since this book is large, your class will probably cover only a portion of its material. Although we hope that you will find this book helpful to you as a course textbook now, we have also tried to make it comprehensive enough to warrant space on your future professional bookshelf.

What are the prerequisites for reading this book?

- You need some programming experience. In particular, you should understand recursive procedures and simple data structures, such as arrays and linked lists (although Section 10.2 covers linked lists and a variant that you may find new).
- You should have some facility with mathematical proofs, and especially proofs by mathematical induction. A few portions of the book rely on some knowledge of elementary calculus. Although this book uses mathematics throughout, Part I and Appendices A–D teach you all the mathematical techniques you will need.

Our website, <http://mitpress.mit.edu/algorithms/>, links to solutions for some of the problems and exercises. Feel free to check your solutions against ours. We ask, however, that you not send your solutions to us.

To the professional

The wide range of topics in this book makes it an excellent handbook on algorithms. Because each chapter is relatively self-contained, you can focus on the topics most relevant to you.

Since most of the algorithms we discuss have great practical utility, we address implementation concerns and other engineering issues. We often provide practical alternatives to the few algorithms that are primarily of theoretical interest.

If you wish to implement any of the algorithms, you should find the translation of our pseudocode into your favorite programming language to be a fairly straightforward task. We have designed the pseudocode to present each algorithm clearly and succinctly. Consequently, we do not address error handling and other software-engineering issues that require specific assumptions about your programming environment. We attempt to present each algorithm simply and directly without allowing the idiosyncrasies of a particular programming language to obscure its essence. If you are used to 0-origin arrays, you might find our frequent practice of

indexing arrays from 1 a minor stumbling block. You can always either subtract 1 from our indices or just overallocate the array and leave position 0 unused.

We understand that if you are using this book outside of a course, then you might be unable to check your solutions to problems and exercises against solutions provided by an instructor. Our website, <http://mitpress.mit.edu/algorithms/>, links to solutions for some of the problems and exercises so that you can check your work. Please do not send your solutions to us.

To our colleagues

We have supplied an extensive bibliography and pointers to the current literature. Each chapter ends with a set of chapter notes that give historical details and references. The chapter notes do not provide a complete reference to the whole field of algorithms, however. Though it may be hard to believe for a book of this size, space constraints prevented us from including many interesting algorithms.

Despite myriad requests from students for solutions to problems and exercises, we have adopted the policy of not citing references for them, removing the temptation for students to look up a solution rather than to discover it themselves.

Changes for the fourth edition

As we said about the changes for the second and third editions, depending on how you look at it, the book changed either not much or quite a bit. A quick look at the table of contents shows that most of the third-edition chapters and sections appear in the fourth edition. We removed three chapters and several sections, but we have added three new chapters and several new sections apart from these new chapters.

We kept the hybrid organization from the first three editions. Rather than organizing chapters only by problem domains or only according to techniques, this book incorporates elements of both. It contains technique-based chapters on divide-and-conquer, dynamic programming, greedy algorithms, amortized analysis, augmenting data structures, NP-completeness, and approximation algorithms. But it also has entire parts on sorting, on data structures for dynamic sets, and on algorithms for graph problems. We find that although you need to know how to apply techniques for designing and analyzing algorithms, problems seldom announce to you which techniques are most amenable to solving them.

Some of the changes in the fourth edition apply generally across the book, and some are specific to particular chapters or sections. Here is a summary of the most significant general changes:

- We added 140 new exercises and 22 new problems. We also improved many of the old exercises and problems, often as the result of reader feedback. (Thanks to all readers who made suggestions.)

- We have color! With designers from the MIT Press, we selected a limited palette, devised to convey information and to be pleasing to the eye. (We are delighted to display red-black trees in—get this—red and black!) To enhance readability, defined terms, pseudocode comments, and page numbers in the index are in color.
- Pseudocode procedures appear on a tan background to make them easier to spot, and they do not necessarily appear on the page of their first reference. When they don't, the text directs you to the relevant page. In the same vein, nonlocal references to numbered equations, theorems, lemmas, and corollaries include the page number.
- We removed topics that were rarely taught. We dropped in their entirety the chapters on Fibonacci heaps, van Emde Boas trees, and computational geometry. In addition, the following material was excised: the maximum-subarray problem, implementing pointers and objects, perfect hashing, randomly built binary search trees, matroids, push-relabel algorithms for maximum flow, the iterative fast Fourier transform method, the details of the simplex algorithm for linear programming, and integer factorization. You can find all the removed material on our website, <http://mitpress.mit.edu/algorithms/>.
- We reviewed the entire book and rewrote sentences, paragraphs, and sections to make the writing clearer, more personal, and gender neutral. For example, the “traveling-salesman problem” in the previous editions is now called the “traveling-salesperson problem.” We believe that it is critically important for engineering and science, including our own field of computer science, to be welcoming to everyone. (The one place that stumped us is in Chapter 13, which requires a term for a parent's sibling. Because the English language has no such gender-neutral term, we regretfully stuck with “uncle.”)
- The chapter notes, bibliography, and index were updated, reflecting the dramatic growth of the field of algorithms since the third edition.
- We corrected errors, posting most corrections on our website of third-edition errata. Those that were reported while we were in full swing preparing this edition were not posted, but were corrected in this edition. (Thanks again to all readers who helped us identify issues.)

The specific changes for the fourth edition include the following:

- We renamed Chapter 3 and added a section giving an overview of asymptotic notation before delving into the formal definitions.
- Chapter 4 underwent substantial changes to improve its mathematical foundation and make it more robust and intuitive. The notion of an algorithmic recurrence was introduced, and the topic of ignoring floors and ceilings in recur-

rences was addressed more rigorously. The second case of the master theorem incorporates polylogarithmic factors, and a rigorous proof of a “continuous” version of the master theorem is now provided. We also present the powerful and general Akra-Bazzi method (without proof).

- The deterministic order-statistic algorithm in Chapter 9 is slightly different, and the analyses of both the randomized and deterministic order-statistic algorithms have been revamped.
- In addition to stacks and queues, Section 10.1 discusses ways to store arrays and matrices.
- Chapter 11 on hash tables includes a modern treatment of hash functions. It also emphasizes linear probing as an efficient method for resolving collisions when the underlying hardware implements caching to favor local searches.
- To replace the sections on matroids in Chapter 15, we converted a problem in the third edition about offline caching into a full section.
- Section 16.4 now contains a more intuitive explanation of the potential functions to analyze table doubling and halving.
- Chapter 17 on augmenting data structures was relocated from Part III to Part V, reflecting our view that this technique goes beyond basic material.
- Chapter 25 is a new chapter about matchings in bipartite graphs. It presents algorithms to find a matching of maximum cardinality, to solve the stable-marriage problem, and to find a maximum-weight matching (known as the “assignment problem”).
- Chapter 26, on task-parallel computing, has been updated with modern terminology, including the name of the chapter.
- Chapter 27, which covers online algorithms, is another new chapter. In an online algorithm, the input arrives over time, rather than being available in its entirety at the start of the algorithm. The chapter describes several examples of online algorithms, including determining how long to wait for an elevator before taking the stairs, maintaining a linked list via the move-to-front heuristic, and evaluating replacement policies for caches.
- In Chapter 29, we removed the detailed presentation of the simplex algorithm, as it was math heavy without really conveying many algorithmic ideas. The chapter now focuses on the key aspect of how to model problems as linear programs, along with the essential duality property of linear programming.
- Section 32.5 adds to the chapter on string matching the simple, yet powerful, structure of suffix arrays.